Chapter 2 Processes and Threads

2.1 Process Concepts

Program

A program is a piece of code which may be a single line or millions of lines. A computer program is usually written by a computer programmer in a programming language. A computer program is a collection of instructions that performs a specific task when executed by a computer. When we compare a program with a process, we can conclude that a process is a dynamic instance of a computer program.

Process

A process is an instance of a program in execution. A program becomes process when executable file is loaded in the main memory. A process is defined as an entity which represents the basic unit of work to be implemented in the system. Each process has its own address space and process control block (PCB).

Process:

- Dynamic
- Part of a program in execution
- a live entity, it can be created, executed and terminated.
- It goes through different states

- Requires resources to be allocated by the OS
- one or more processes may be executing the same code.

Program:

- static
- no states

This example illustrate the difference between a process and a program:

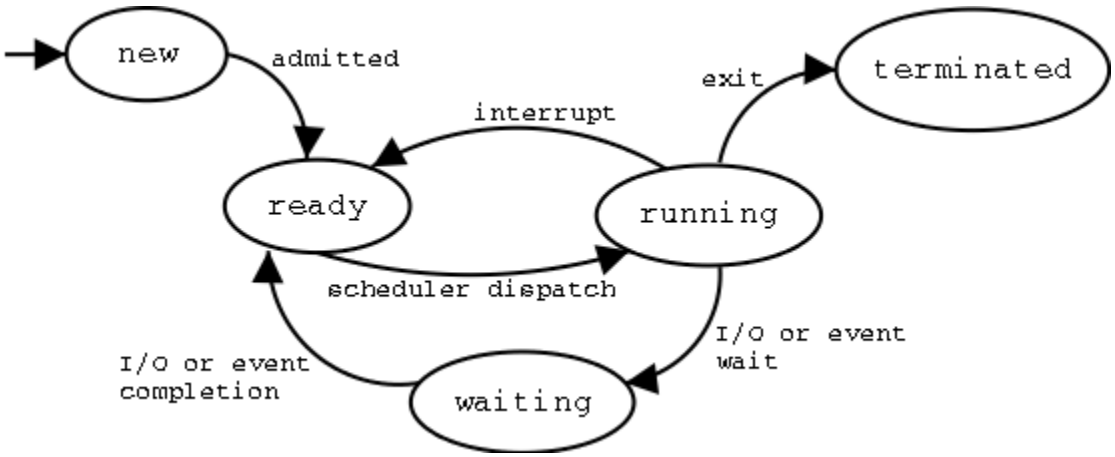
```
main ()
{
        int i , prod =1;
        for (i=0;i<100;i++)
                        prod =
                        pord*i;
}
```

It is a program containing one multiplication statement (prod = prod * i) but the process will execute100 multiplication, one at a time through the 'for' loop.

Although two processes may be associated with the same program, they are nevertheless considered two separate execution sequences. For instance several users may be running different copies of mail program, or the same user may invoke many copies of web browser program. Each of these is a separate process, and although the text sections are equivalent, the data, heap and stack section may vary.

Process states and its transition

When a process executes, it passes through different states. These stages may differ in different operating systems, and the names of these states are also not standardized. In general, a process can have one of the following five states at a time.



Start: This is the initial state when a process is first started/created.

Ready: The process is waiting to be assigned to a processor. Ready processes are waiting to have the processor allocated to them by the operating system so that they can run. Process may come into this state after Start state or while running it by but interrupted by the scheduler to assign CPU to some other process.

Running: Once the process has been assigned to a processor by the OS scheduler, the process state is set to running and the processor executes its instructions.

Waiting: Process moves into the waiting state if it needs to wait for a resource, such as waiting for user input, or waiting for a file to become available.

Terminated or Exit: Once the process finishes its execution, or it is terminated by the operating system, it is moved to the terminated state where it waits to be removed from main memory.

Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminates. A PCB keeps all the information needed to keep track of a process as listed in the figure:

Process ID: Unique identification for each of the process in the operating system.

- **Process State**: The current state of the process i.e., whether it is ready, running, waiting, or whatever.
- **Pointer**: A pointer to parent process.
- **Priority:** Contains the priority numbers of each process.
- **Program Counter**: Program Counter is a pointer to the address of the next instruction to be executed for this process.

- **CPU registers**: Various CPU registers where process need to be stored for execution for running state.
- **IO status information**: This includes a list of I/O devices allocated to the process.
- **Accounting information**: This includes the amount of CPU used for process execution, time limits, execution ID etc
- **CPU Scheduling Information**: Process priority and other scheduling information which is required to schedule the process.
- **Memory management information**: This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.

Operation of process

The operations of process carried out by an operating system are primarily of two types:

i. Process Creation

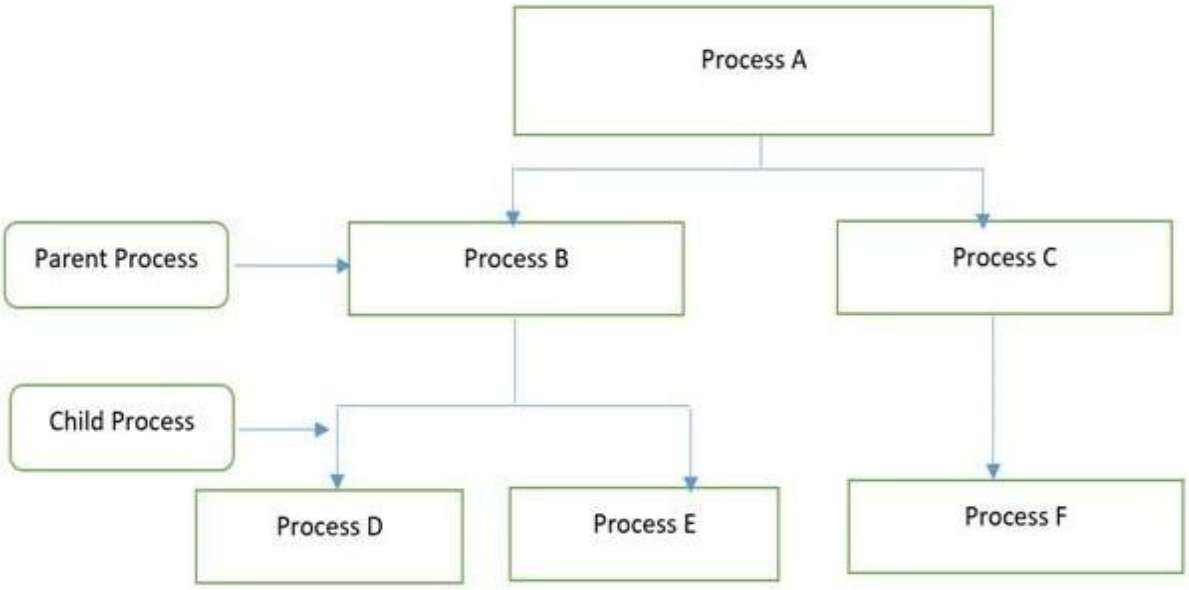
Process creation is a task of creating new processes. There are different situations in which a new process is created. There are different ways to create new process. A new process can be created at the time of initialization of operating system or when system calls such as **fork ()** are initiated by other processes. The process, which creates a new process using system calls, is called parent process while the new process that is created is called child process. The child processes can create new processes using system calls. A new process can also create by an operating system based on the request received from the user.

The process creation is very common in running computer system because corresponding to every task that is performed there is a process associated with it. For instance, a new process is created every time a user logs on to a computer system, an application program such a MS Word is initiated, or when a document printed.

ii. Process termination

Process termination is an operation in which a process is terminated after the execution of its last instruction. This operation is used to terminate or end any process. When a process is terminated, the resources that were being utilized by the process are released by the operating system. When a child process terminates, it sends the status information back to the parent process before terminating. The child process can also be terminated by the parent process if the task performed by the child process is no longer needed. In addition, when a parent process terminates, it has to terminate the child process as well became a child process cannot run when its parent process has been terminated.

The termination of a process when all its instruction has been executed successfully is called normal termination. However, there are instances when a process terminates due to some error. This termination is called as abnormal termination of a process.



Concurrent processing

Concurrent processing is a computing model in which multiple processors execute instructions simultaneously for better performance. Tasks are broken down into subtasks that are then assigned to separate processors to perform simultaneously, instead of sequentially as they would have to be carried out by a single processor. Concurrent processing is sometimes said to be synonymous with parallel processing.

Parallel Processing

Parallel processing, one form of multiprocessing, is a situation in which two or more processors operate in unison. Two or more CPUs are executing instructions simultaneously. The Processor Manager has to coordinate the activity of each processor as well as synchronize cooperative interaction among the CPUs.

There are two primary benefits to parallel processing systems:

- **Increased reliability**

✓ The availability of more than one CPU

✓ If one processor fails, then the others can continue to operate and absorb the load.

- **Faster processing**

The increased processing speed is often achieved because sometimes Instructions can be processed in parallel, two or more at a time in one of several ways:

✓ Some systems allocate a CPU to each program or job

✓ Others allocate a CPU to each working set or parts of it

✓ Others subdivide individual instructions so that each subdivision can be processed simultaneously.

Race Condition

A situation where several processes/threads access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place, is called a race condition. A race condition occurs when two or more threads can access shared data and they try to change it at the same time.

Possibilities of race:

- ✓ many concurrent process **read the same data**.
- ✓ one process **reading** and another process **writing** same data.
- ✓ two or more process **writing same data**. **Example 1**
- ✓ P1 and P2 share global variables b and c with initial values b=1 and c=2
- ✓ P1 has an assignment operation, b=b+c
- ✓ P2 has an assignment operation c=b+c
- ✓ If P1 an P2 are running concurrently, value of b and c would be 3 and 5 if P1 executes its assignment first or it could be 4 and 3 if P2 executes its assignment first.

Example 2

Single process	output
Read(X)	**X=5**
X= X + 5	**X=10**

Case 1:

Process 1	Process 2	X
Read(x)		5
X=X+5		10
	Read(X)	10
	X=X+5	15

Case 2:

Process 1	Process 2	X
	Read(x)	5
	X=X+5	10
Read(x)		10
X=X+5		15

Case 3:

Process 1	Process 2	X
Read(x)		5
	Read(x)	5
X=X+5		10
	X=X+5	10

To guard against the race conditions, we need to synchronize the processes in such a way that only one process at a time can be manipulating shared variable or file. This is called mutual exclusion (i.e. critical section is executed as an atomic instruction).

2.2 Inter-process Communication

IPC is the mechanism for processes to communicate and to synchronize their actions or to allow processes to exchange information.

There are numerous reasons for providing an environment or situation which allows process co- operation:

Information sharing: Since some users may be interested in the same piece of information (for example, a shared file), you must provide a situation for allowing concurrent access to that information.

Computation speedup: If you want a particular work to run fast, you must break it into sub- tasks where each of them will get executed in parallel with the other tasks. Note that such a speed-up can be attained only when the computer has compound or various processing elements like CPUs or I/O channels.

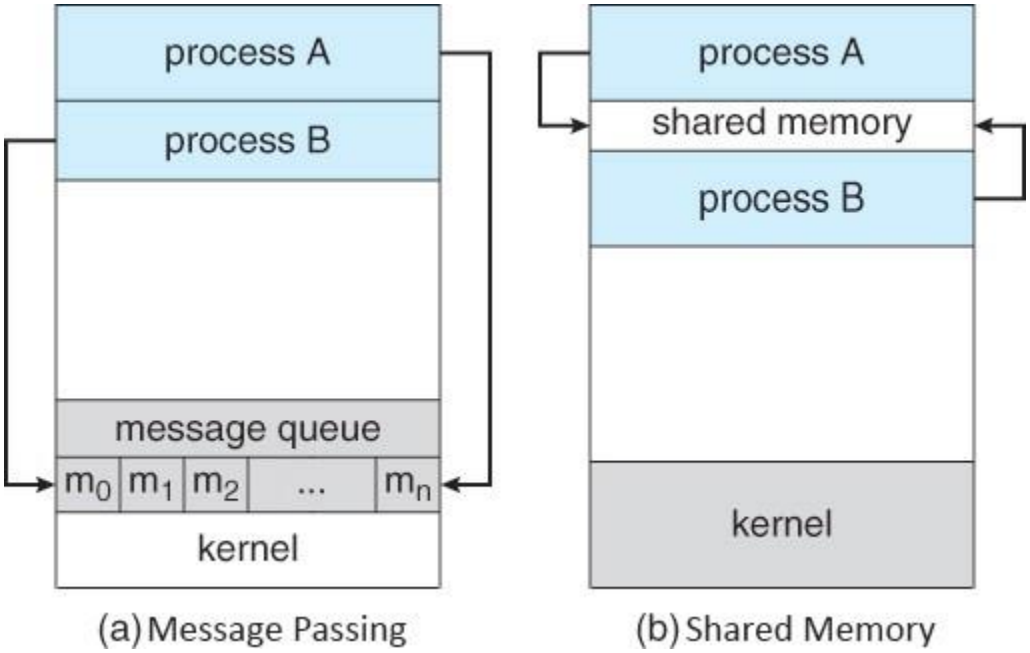
Modularity: You may want to build the system in a modular way by dividing the system functions into split processes or threads.

Convenience: Even a single user may work on many tasks at a time. For example, a user may be editing, formatting, printing, and compiling in parallel.

Working together with multiple processes, require an inter-process communication (IPC) method which will allow them to exchange data along with various information. There are two primary models of inter-process communication:

i. shared memory and

ii. message passing.



Shared Memory:

- Here a region of memory that is shared by co-operating process is established.
- Process can exchange the information by reading and writing data to the shared region.
- Shared memory allows maximum speed and convenience of communication as it can be done at the speed of memory within the computer.
- System calls are required only to establish shared memory regions. Once shared memory is established no assistance from the kernel is required,all access are treated as routine memory access.

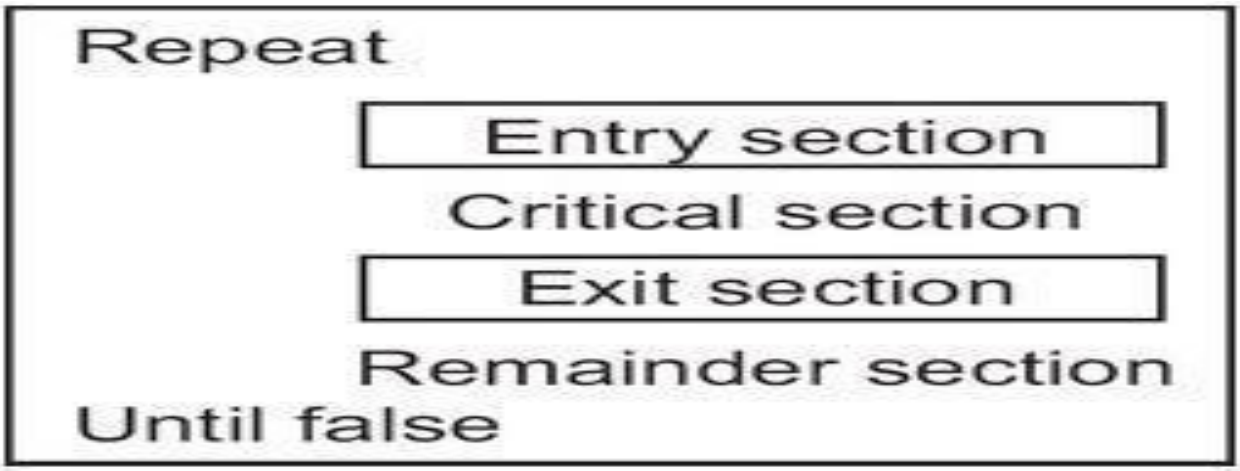
Message Passing:

- communication takes place by means of messages exchanged between the co-operating process
- Message passing is useful for exchanging the smaller amount of data since no conflict need to be avoided.
- Easier to implement than shared memory.
- Slower than that of Shared memory as message passing system are typically implemented using system call which requires more time consuming task of Kernel intervnetion.

Critical Section/region

- Critical section is the area of the code/instruction that should be executed atomically.
- Critical section is a piece of code that accesses a shared resource (data structure or device) that must not be concurrently accessed by more than one thread of execution.

- If a process wants to access any kind of common shareable variable, common shareable table, common shareable files in its then the process is trying to enter a critical section.



Requirements for Critical Section

1. Mutual Exclusion

At a time only one process can enter/ access to the critical section.

2. Progress

If CS is vacant/free and some process is tried to enter CS, no **infinite time** should be taken for the selection of process to go into the CS. Processes on the remainder section also get chance to enter CS.

3. Bounded Waiting

Let a particular process request to enter a CS, by OS allowing other process to enter CS. In such case there must be upper bound that the process gets denied to enter CS.

Avoiding Race Conditions:

Critical Section:

To avoid race condition we need **Mutual Exclusion. Mutual Exclusion** is someway of making sure that if one process is using a shared variable or file, the other processes will be excluded from doing the same things.

The difficulty above in the printer spooler occurs because process B started using one of the shared variables before process A was finished with it.

That part of the program where the shared memory is accessed is called the **critical region or critical section**. If we could arrange matters such that no two processes were ever in their critical regions at the

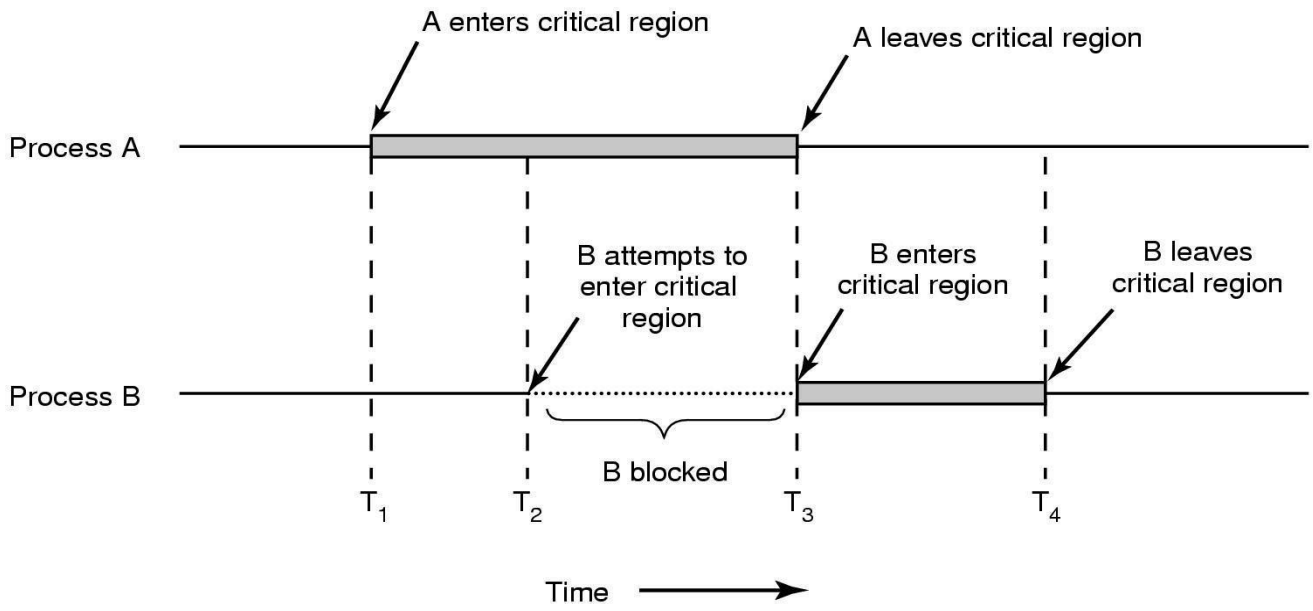
same time, we could avoid race conditions. Although this requirement avoids race conditions, this is not sufficient for having parallel processes cooperate correctly and efficiently using shared data.

(Rules for avoiding Race Condition) **<u>Solution to Critical section problem:</u>**

1. **No two processes may be simultaneously inside their critical regions. (Mutual Exclusion)**
2. **No assumptions may be made about speeds or the number of CPUs.**
3. **No process running outside its critical region may block other processes.**
4. **No process should have to wait forever to enter its critical region.**

Mutual Exclusion

- A mutual exclusion (mutex) is a program object that prevents simultaneous access to a shared resource.
- Mutual exclusion is a property of concurrency control, which is instituted for the purpose of preventing race conditions.
- It is the requirement that one thread of execution never enter its critical section at the same time that another concurrent thread of execution enters its own critical section.



.

i. <u>Mutual Exclusion with busy waiting</u>

```
while (TRUE) {
    while (turn != 0)        /* loop */ ;
    critical_region( );
    turn = 1;
    noncritical_region( );
}
```

(a)

Process 1

```
while (TRUE) {
    while (turn != 1)        /* loop */ ;
    critical_region( );
    turn = 0;
    noncritical_region( );
}
```

(b)

Process 2

- In this proposed solution, the integer variable 'turn' keeps track of whose turn is to enter the critical section.

- Initially turn = 0. Process 1 inspects turn, finds it to be 0 and enters in its critical section. Process 2 also finds it to be 0 and therefore sits in a loop continually testing 'turn' to see when it becomes 1.
- Continuously testing a variable waiting for some value to appear is called the **busy waiting**.
- When process 1 exit from the critical region is set to turn to 1 and now process 2 can find it to be 1 and enters into the critical region.
- Also, in this way both the process get the alternate turn to enter in the critical region.
- Taking turns is not a good idea when one of the processes is much slower than the other.
- Can used only when the waiting period is expected to be short.

Mutual Exclusion can be achieved using

i.Semaphore

- Semaphore is a protected integer value that can facilitate the processes to enter to a critical section.
- It ensures only one process enters to a CS at a time in a mutually exclusive way.
- Perform **atomic operation** i.e. no partial execution

Operations

i. **wait(s)**

```
while  (s = = 0 );     do no operation
s=s-1
```

ii. signal(s)

```
s=s+1
```

Semaphore is a simply a variable. This variable is used to solve critical section problem and to achieve process synchronization in the multi-processing environment. The two most common kinds of semaphores are counting semaphores and binary semaphores. Counting semaphore can take non-negative integer values and Binary semaphore can take the value 0 & 1. only.

Now let us see how it do so. First look at two operations which can be used to access and change the value of semaphore variable.

```
P(Semaphore s){
    while(S == 0);    /* wait until s=0 */
  s=s-1;
}

V(Semaphore s){
          s=s+1;
}
```

Note that there is Semicolon after while. The code gets stuck Here while s is 0.

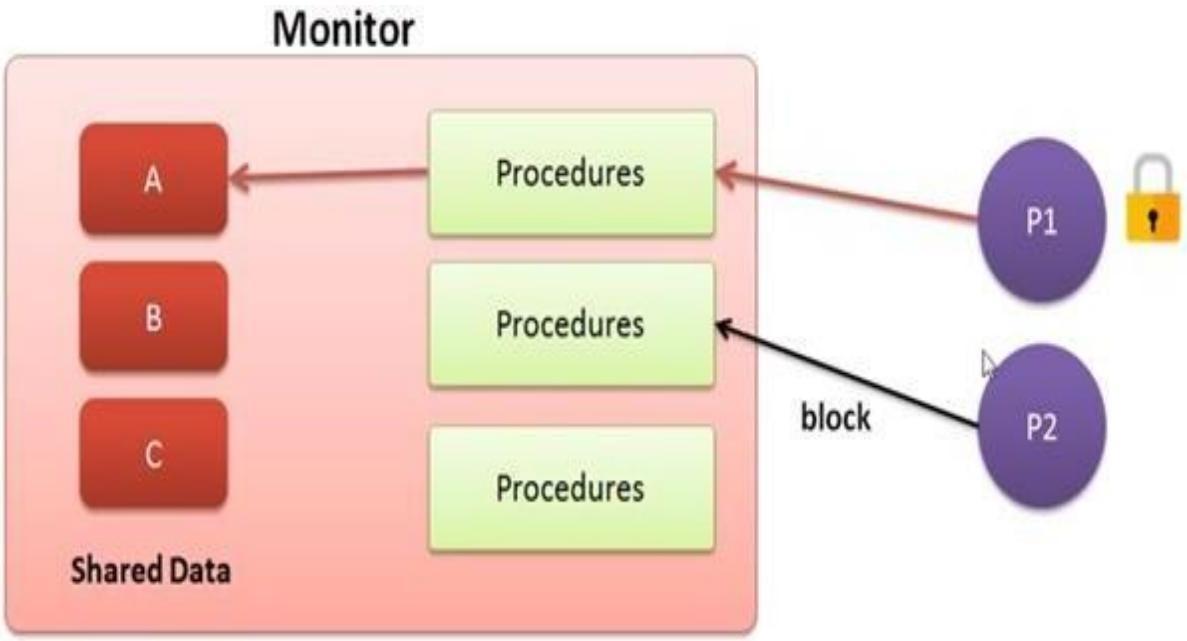
Some point regarding P and V operation

1. P operation is also called wait, sleep or down operation and V operation is also called signal, wake-up or up operation.
2. Both operations are atomic and semaphore(s) is always initialized to one.
3. A critical section is surrounded by both operations to implement process synchronization. See below image. Critical section of Process P is in between P and V operation.

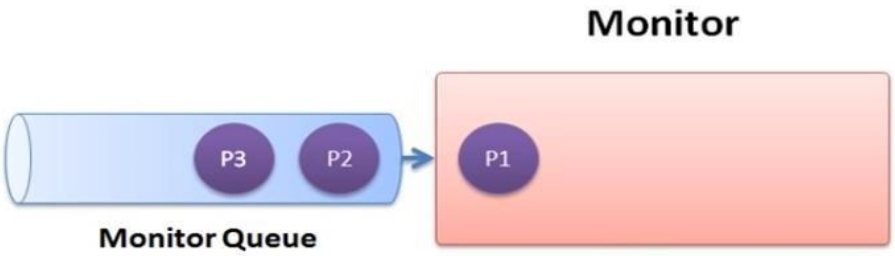
```
Process P

// Some code
P(s);
  // critical section
V(s);
  // remainder section
```

ii.Monitor

- A monitor is a collection of procedures, variables and data structures that are grouped together in a special kind of module or package
- Once the process enters in monitor, the process gets locked and remaining process are kept in queue
- Monitor is a type, or abstract data type, encapsulates private data with public methods to operate on that data



- Processes may call the procedures in a monitor whenever they want to but they cannot directly access the monitor's internal data structures from procedures declared outside the monitor



✓ **Serializability**

Serializability in operating systems, particularly in the context of **transaction management** in databases, refers to ensuring that the execution of multiple transactions concurrently results in the same state as if the transactions were executed sequentially (one after another). This is crucial for maintaining **data consistency** in a multi-user or multi-process environment.

Key Concepts of Serializability:

1. **Transactions**: A sequence of operations performed as a single logical unit of work. Transactions must adhere to the **ACID properties** (Atomicity, Consistency, Isolation, Durability).
2. **Concurrency**: In real-world systems, multiple transactions often execute simultaneously to improve performance and resource utilization. However, this can lead to conflicts or inconsistencies if not managed properly.
3. **Serial Schedule**: A schedule (order of execution of transactions) where transactions are executed one after the other, without overlapping.
4. **Serializable Schedule**: A schedule is considered serializable if its result is equivalent to some serial schedule, even though transactions may execute concurrently.

Types of Serializability:

1. **Conflict Serializability**:

   Focuses on conflicts between operations (e.g., read/write or write/write conflicts).

   A schedule is conflict-serializable if it can be transformed into a serial schedule by reordering non-conflicting operations.

   Uses a **precedence graph (dependency graph)** to check for cycles. If there's no cycle, the schedule is conflict-serializable.

2. **View Serializability**:

   A schedule is view-serializable if it produces the same final outcome as a serial schedule.

   Less strict than conflict serializability but harder to enforce.

Ensuring Serializability:

1. **Locking Protocols**:

   Two-Phase Locking (2PL) ensures conflict serializability by dividing the transaction into two phases:

   Growing phase: Acquires locks but doesn't release them.

Shrinking phase: Releases locks but doesn't acquire new ones.

2. **Timestamp Ordering**:

   Assigns a unique timestamp to each transaction to determine the order of operations.

3. **Validation Protocols**:

   Validate transactions at the commit phase to ensure no conflicts have occurred.

Importance of Serializability:

- Prevents **data corruption** or **inconsistencies** due to concurrent transaction execution.
- Maintains **isolation**, a key aspect of the ACID properties.
- Ensures reliability in systems like databases, distributed systems, and transaction-processing systems.

Techniques for avoiding Race Condition:

1. Disabling Interrupts
2. Lock Variables
3. Strict Alteration
4. Peterson's Solution
5. TSL instruction
6. Sleep and Wakeup
7. Semaphores
8. Monitors
9. Message Passing

1. Disabling Interrupts

Disabling interrupts is a low-level mechanism to ensure that the CPU is not interrupted while executing a critical section of code. This guarantees that no other process or interrupt handler can access shared resources during this period.

- The CPU has an interrupt-enable flag.
- Disabling interrupts sets this flag to prevent the CPU from responding to any external or internal interrupts.
- The critical section executes without being preempted.
- After the critical section completes, interrupts are re-enabled.

Steps:

1. Disable interrupts before entering the critical section.
2. Execute the critical section.
3. Re-enable interrupts after leaving the critical section.

Example (Pseudo-code):

```
disable_interrupts();   // Prevent interruptions
critical_section();     // Access shared resource
```

enable_interrupts(); // Re-enable interruptions

2. Lock Variable

Lock variables are a higher-level software mechanism used to coordinate access to a shared resource between multiple processes or threads. A lock ensures that only one process or thread can access the critical section at any given time.

How it Works

- A shared variable (the lock) is used to indicate the state of the critical section:
  - **0 (unlocked)**: No process is in the critical section.
  - **1 (locked)**: A process is in the critical section.
- Before entering the critical section, a process checks the lock:
  - If unlocked, it sets the lock and proceeds.
  - If locked, it waits until the lock is released.

Steps:

1. Check the value of the lock variable.
2. If it is 0, set it to 1 (lock it) and enter the critical section.
3. After exiting the critical section, reset the lock to 0 (unlock it).

Example (Pseudo-code):

```
while (lock == 1) {
   // Busy wait until the lock is free
}
lock = 1;            // Acquire the lock
critical_section();  // Access shared resource
lock = 0;            // Release the lock
```

Disadvantages:

1. **Busy Waiting**:
   - Processes may waste CPU cycles in a spinlock (polling the lock).
2. **Non-Atomicity**:

- Checking and setting the lock value is not atomic, leading to potential race conditions. For example, two processes may check the lock simultaneously, both find it free, and enter the critical section.

3. Strict Alternation

- In this approach, we have an integer variable named turn, which keeps track of whose turn is it to enter critical section
- Say, in a two process scenario, Process 0 runs when turn=0 and process 1 runs when turn=1

```
while(TRUE){while(TRUE){ while(turn!=0);
critical_region();
turn=1;
noncritical_region();
}
```

Fig: Process 0

```
while(turn!=1);
critical_region();
turn=0;
noncritical_region();
}
```

Fig: Process 1

- Turn is initially set to 0
- Process 0 examines turn to be 0 and enters the critical region.

- Process 1 also finds turn to be 0 so it sits in a tight loop continually testing turn to see when it becomes 1. Continuously testing a variable until some value appears is called busy waiting
- When process 0 leaves the critical region, it setsturn to 1 so that process 1 can now enter the critical region.

Problem

If process 0 finishes its work in non-critical region fast enough after it set, turn to 1 and process 0needs to enter critical section again before process 1 then it cannot do so. So, process 1 which is running outside critical region is blocking process 0.

4. Peterson's Algorithm

Peterson's Algorithm is a simple and elegant solution for achieving **mutual exclusion** between two processes. It overcomes many limitations of simpler techniques like strict alternation. This algorithm satisfies all the requirements for mutual exclusion, including **progress**, **bounded waiting**, and **mutual exclusion** itself.

Peterson's solution uses:

1. **A flag array** to indicate whether a process is interested in entering the critical section.
2. **A turn variable** to decide which process gets priority when both want to enter the critical section.

By combining these two mechanisms, Peterson's algorithm ensures that only one process can enter the critical section at a time.

Algorithm Details

Shared Variables:

- flag[2]: An array where:
  - flag[0] is true if Process P0P_0P0 wants to enter the critical section.
  - flag[1] is true if Process P1P_1P1 wants to enter the critical section.
- turn: A variable indicating whose turn it is to enter the critical section when both processes are interested.

For process p0

```
while (true) {
   flag[0] = true;      // Indicate interest
   turn = 1;            // Give priority to P1
   while (flag[1] && turn == 1); // Wait if P1 is interested and has priority

   critical_section();   // Access shared resource

   flag[0] = false;      // Indicate done
   remainder_section();  // Non-critical work
}
```

Process P1 :

```
while (true) {
```

```
    flag[1] = true;       // Indicate interest
    turn = 0;             // Give priority to P0
    while (flag[0] && turn == 0); // Wait if P0 is interested and has priority

    critical_section();   // Access shared resource

    flag[1] = false;      // Indicate done
    remainder_section();  // Non-critical work
}
```

Properties of Peterson's Algorithm

1. Mutual Exclusion:

- At most, one process can enter the critical section because the waiting condition ensures that the other process cannot proceed simultaneously.

2. Progress:

- If one process is not interested in entering the critical section (flag[i] == false), the other process does not have to wait.

3. Bounded Waiting:

- Each process gets a fair chance to enter the critical section because the turn variable alternates priority.

4. Deadlock-Free:

- There are no conditions where both processes are indefinitely blocked.

5. Starvation-Free:

- Every interested process eventually enters the critical section.

Disadvantages

1. Limited to two processes.
2. Not practical in modern multi-processor systems or when there are more than two processes.
3. Relies on busy waiting, which can waste CPU cycles.

5 . Dekker's Algorithm

Dekker's Algorithm is one of the earliest solutions to the **mutual exclusion problem** in concurrent programming. It allows two processes to share a single-use resource without conflicts, ensuring mutual exclusion, progress, and bounded waiting.

Dekker's Algorithm was the first correct solution to mutual exclusion for two processes, relying only on shared memory and no special hardware instructions.

Dekker's Algorithm uses:

1. **A flag array**: Each process indicates its interest in entering the critical section.
2. **A turn variable**: Determines which process gets priority if both want to enter the critical section.

It ensures mutual exclusion by carefully managing these variables, alternating access to the critical section when both processes want it simultaneously.

Shared Variables:

- flag[2]: An array where:
  - flag[0] = true: Process P0 wants to enter the critical section.
  - flag[1] = true: Process P1 wants to enter the critical section.
- turn: Indicates whose turn it is to access the critical section when both processes are interested.

For process p0

```
while (true) {
   flag[0] = true;          // Indicate interest in the critical section
   while (flag[1]) {        // Check if P1 is interested
      if (turn == 1) {      // If it's P1's turn
         flag[0] = false; // Relinquish interest
         while (turn == 1); // Wait until P1's turn is over
         flag[0] = true;  // Reassert interest
      }
   }
   critical_section();      // Enter the critical section
   turn = 1;                // Give turn to P1
   flag[0] = false;         // Release interest
   remainder_section();     // Perform non-critical work
```

```
}
```

FOR process p1

```
while (true) {
    flag[1] = true;          // Indicate interest in the critical section
    while (flag[0]) {        // Check if P0 is interested
        if (turn == 0) {     // If it's P0's turn
            flag[1] = false; // Relinquish interest
            while (turn == 0); // Wait until P0's turn is over
            flag[1] = true;  // Reassert interest
        }
    }
    critical_section();      // Enter the critical section
    turn = 0;                // Give turn to P0
    flag[1] = false;         // Release interest
    remainder_section();     // Perform non-critical work
}
```

6.Test-and-Set Lock

Test-and-Set Lock (TSL) is a hardware-supported instruction used to implement mutual exclusion in multi-process systems. It ensures atomicity, making it a powerful primitive for synchronization.

The **Test-and-Set** instruction atomically performs the following actions:

1. Reads the value of a memory location (e.g., a lock variable).
2. Sets the value of that memory location to true (locked).

The atomicity ensures that no other process can interfere during the read-and-set operation, making it suitable for implementing locks

How TSL Works

1. **Shared Lock Variable**:
   - A shared boolean variable lock is used:
     - false: The lock is free.
     - true: The lock is taken.
2. **Test-and-Set Instruction**:
   - Syntax (in pseudo-assembly):

1.TEST_AND_SET(register, lock)

- register: Temporary register to store the old value of lock.
- lock: The memory location to be modified.

2. Operation:

```
old_value = lock;
lock = true;
return old_value;
```

3. **Critical Section Entry**:
   - A process repeatedly invokes TEST_AND_SET in a loop until it gets the lock (lock == false).

Code

```
while (true) {
   // Entry section
   while (TEST_AND_SET(&lock)); // Busy-wait if the lock is already taken

   // Critical section
   critical_section();
```

```
    // Exit section
    lock = false; // Release the lock

    // Remainder section
    remainder_section();
}
```

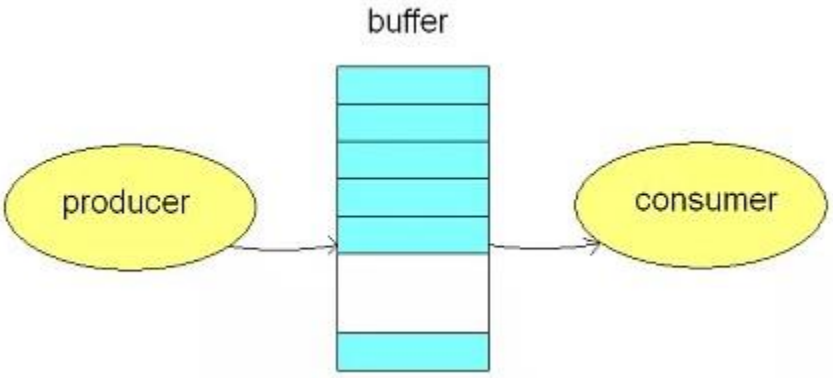
7. Producer-consumer problem

Producer-consumer problem (also known as the bounded-buffer problem) is a multiprocessing synchronization problem.

The problem describes two processes, the producer and the consumer, who share a common, fixedsize buffer.

Producer: - The producer's job is to generate a piece of data, put it into the buffer and start again.

Consumer: - The consumer is consuming the data (i.e., removing it from the buffer) one piece ata time.



Problem

The problem is to make sure that the producer won't try to add data into the buffer if it's full andthat the consumer won't try to remove data from an empty buffer.

Solution to Producer-consumer problem

Producer either go to sleep or discard data if the buffer is full. The next time the consumer removesan item from the buffer and it notifies the producer who starts to fill the buffer again consumer can go to sleep if it finds the buffer to be empty. The next time the producer puts data into the buffer, it wakes up the sleeping consumer.

Sleep and wakeup

- When a process wants to enter its critical region, it checks to see if entry is allowed, if it is not, the process just sits in a tight loop waiting, this approach wastes CPU time.
- We could use an IPC primitives that blocks a process instead of wasting CPU time when they are not allowed to enter into their critical section.
- Such a simple primitive pair is **sleep and wakeup**
- Essentially, when a process is not permitted to access its critical section, it uses a system call known as Sleep, which causes that process to block.
- The process will not be scheduled to run again, until another process uses the Wakeup system call.
- Wakeup is called by a process when it leaves its critical section if any other processes have blocked.

```
BufferSize = 3;
    count = 0;
    producer()
{
int item;
WHILE (true){
make_new(item);                          // create a new item to put in the buffer
                IF(count==BufferSize) Sleep();   // if the buffer is full, sleep
                put_item(item);          // put the item in the buffer
count = count + 1;                       // increment count of items
                IF (count==1)
Wakeup(Consumer);                        // if the buffer was previously empty, wake the
                consumer
} }
Consumer()
{
Int item;
WHILE(true)
{
IF(count==0) Sleep();                    // if the buffer is empty, sleep
remove_item(item);                       // take an item from the buffer
count = count - 1;                        // decrement count of items
                IF(count==N-1)
Wakeup(Producer);                         // if buffer was previously full, wake the producer
Consume_item(item);                      // consume the item
}}
```

Problem with the above Solution

The problem with this solution is that it contains a race condition that can lead into a deadlock. Consider the following scenario:

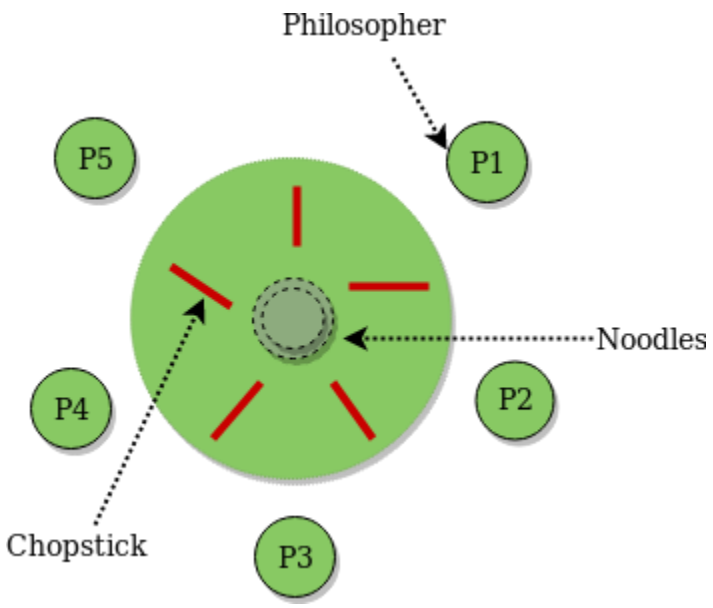
- ✓ The consumer has just read the variable itemCount, noticed it's zero and is just about to move inside the if-block.
- ✓ Just before calling sleep, the consumer is interrupted and the producer is resumed.
- ✓ The producer creates an item, puts it into the buffer, and increases itemCount.
- ✓ Because the buffer was empty prior to the last addition, the producer tries to wake up the consumer.
- ✓ Unfortunately, the consumer wasn't yet sleeping, and the wakeup call is lost. When the consumer resumes, it goes to sleep and will never be awakened again. This is because the consumer is only awakened by the producer when itemCount is equal to 1.
- ✓ The producer will loop until the buffer is full, after which it will also go to sleep.
- ✓ Since both processes will sleep forever, we have run into a deadlock. This solution therefore is unsatisfactory.

Classical IPC problems

i. The Dining Philosophers Problem

The dining philosophers' problem is a classic synchronization problem which is used to evaluate situations where there is a need of allocating multiple resources to multiple processes**.**

Consider there are five philosophers sitting around a circular dining table. The dining table has five chopsticks and a bowl of noodles in the middle as shown in the below figure. At any instant, a philosopher is either eating or thinking. When a philosopher wants to eat, he uses two chopsticks - one from their left and one from their right. When a philosopher wants to think, he keeps down both chopsticks at their original place.



From the problem statement, it is clear that a philosopher can think for an indefinite amount of time. But when a philosopher starts eating, he has to stop at some point of time. The philosopher is in an endless cycle of thinking and eating.

An array of five semaphores, stick[5], for each of the five chopsticks.The code for each philosopher looks like:

```
while(TRUE)
{
wait(stick[i]);
/*
mod is used because if i=5, next chopstick is 1
(dining table is circular)
*/
wait(stick[(i+1) % 5]);
/* eat */
signal(st
ick[i]);
signal(stick[(i+1) % 5]);
/* think */}
```

When a philosopher wants to eat the rice, he will wait for the chopstick at his left and picks up that chopstick. Then he waits for the right chopstick to be available, and then picks it too. After eating,he puts both the chopsticks down.

But if all five philosophers are hungry simultaneously, and each of them pickup one chopstick, then a deadlock situation occurs because they will be waiting for another chopstick forever. The possible solutions for this are:

- A philosopher must be allowed to pick up the chopsticks only if both the left and right chopsticks are available.
- Allow only four philosophers to sit at the table. That way, if all the four philosophers pick up four chopsticks, there will be one chopstick left on the table. So, one philosopher can start eating and eventually, two chopsticks will be available. In this way, deadlocks can beavoided.

ii. Readers and writers problem

Consider a situation where we have a file shared between many people.

If one of the people tries editing the file, no other person should be reading or writing atthe same time, otherwise changes will not be visible to him/her.
However, if some person is reading the file, then others may read it at the same time.
Precisely in OS we call this situation as the readers-writers problem.

Problem parameters:

One set of data is shared among a number of processes
Once a writer is ready, it performs its write. Only one writer may write at a time
If a process is writing, no other process can read it
If at least one reader is reading, no other process can write
Readers may not write and only read

Solution when Reader has the Priority over Writer

Here priority means, no reader should wait if the share is currently opened for reading.hree variables are used: mutex, wrt, readcnt to implement solution

semaphore mutex, wrt; // semaphore mutex is used to ensure mutual exclusion when readcnt is updated i.e. when any reader enters or exit from the critical section andsemaphore wrt is used by both readers and writers

int readcnt; // readcnt tells the number of processes performing read in the critical section, initially 0

Functions for sempahore :

wait() : decrements the semaphore value.
signal() : increments the semaphore value.

Writer process:

1. Writer requests the entry to critical section.

2. If allowed i.e. wait() gives a true value, it enters and performs the write. If not allowed, it keeps on waiting.
3. It exits the critical section.

```
do {
// writer requests for critical section
wait(wrt);
// performs the write
// leaves the critical section
signal(wrt);
} while(true);
```

Reader process:

Reader requests the entry to critical section.

If allowed:

it increments the count of number of readers inside the critical section. If this reader isthe first reader entering, it locks the wrt semaphore to restrict the entry of writers if anyreader is inside.

It then, signals mutex as any other reader is allowed to enter while others are already reading.

- After performing reading, it exits the critical section. When exiting, it checks if no more reader is inside, it signals the semaphore "wrt" as now, writer can enter the critical section.

2. If not allowed, it keeps on waiting.

```
do {
// Reader wants to enter the critical sectionwait(mutex);
// The number of readers has now increased by 1readcnt++;
// there is atleast one reader in the critical section
// this ensure no writer can enter if there is even one reader
// thus we give preference to readers hereif (readcnt==1)
wait(wrt);
// other readers can enter while this current reader is inside
// the critical sectionsignal(mutex);
// current reader performs reading here wait(mutex); //
                a reader wants to leavereadcnt--;
// that is, no reader is left in the critical section,if (readcnt
                == 0)
signal(wrt);    // writers can entersignal(mutex); //
                reader leaves
} while(true);
```

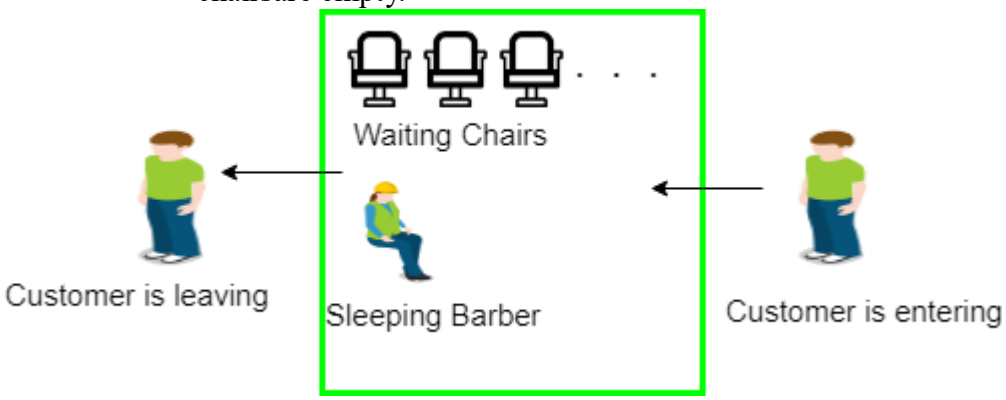
Thus, the semaphore 'wrt' is queued on both readers and writers in a manner such that preferenceis given to readers if writers are also there. Thus, no reader is waiting simply because a writer has requested to enter the critical section.

iii. Sleeping Barber problem

Problem: The analogy is based upon a hypothetical barber shop with one barber. There is a barbershop which has one barber, one barber chair, and n chairs for waiting for customers if there are any to sit on the chair.

- If there is no customer, then the barber sleeps in his own chair.
- When a customer arrives, he has to wake up the barber.

- If there are many customers and the barber is cutting a customer's hair, then the remaining customers either wait if there are empty chairs in the waiting room or they leave if no chairsare empty.



Solution: The solution to this problem includes three semaphores. First is for the customer whichcounts the number of customers present in the waiting room (customer in the barber chair is not included because he is not waiting). Second, the barber 0 or 1 is used to tell whether the barber isidle or is working, And the third mutex is used to provide the mutual exclusion which is requiredfor the process to execute. In the solution, the customer has the record of the number of customerswaiting in the waiting room if the number of customers is equal to the number of chairs in the waiting room then the upcoming customer leaves the barbershop.

When the barber shows up in the morning, he executes the procedure barber, causing him to blockon the semaphore customers because it is initially 0. Then the barber goes to sleep until the first customer comes up.

When a customer arrives, he executes customer procedure the customer acquires the mutex for entering the critical region, if another customer enters thereafter, the second one will not be able to anything until the first one has released the mutex. The customer then checks the chairs in the waiting room if waiting customers are less then the number of chairs then he sits otherwise he leaves and releases the mutex.

If the chair is available then customer sits in the waiting room and increments the variable waitingvalue and also increases the customer's semaphore this wakes up the barber if he is sleeping.

At this point, customer and barber are both awake and the barber is ready to give that person a haircut. When the haircut is over, the customer exits the procedure and if there are no customers in waiting room barber sleeps.

```
Semaphore customer = 0;  // Initially, no customers are waiting.
Semaphore barber = 0;    // Barber starts sleeping.
Semaphore mutex = 1;     // Mutex for mutual exclusion.
int waiting = 0;         // Number of customers waiting in the waiting room.
int max_waiting = N;     // Maximum number of waiting chairs (fixed number).
```

Barber process

```
while (true) {
    wait(customer);         // Wait for a customer to arrive (or be signaled).
    wait(mutex);            // Ensure mutual exclusion to modify the waiting count.
    waiting--;              // Decrease the number of waiting customers.
    signal(barber);         // Wake up the barber to start cutting hair.
    signal(mutex);          // Release mutex after modifying waiting count.

    cut_hair();             // Barber cuts hair.
}
```

Customer process

```
while (true) {
    wait(mutex);            // Ensure mutual exclusion to check waiting count.

    if (waiting < max_waiting) {  // There is space in the waiting room.
        waiting++;          // Increment the number of waiting customers.
        signal(customer);   // Signal the barber that a customer is waiting.
        signal(mutex);      // Release mutex after modifying waiting count.

        wait(barber);       // Wait for the barber to be ready.
```

```
    get_haircut();      // Get the haircut.
} else {                // No space in the waiting room.
    signal(mutex);      // Release mutex if there's no space.
    leave_shop();       // The customer leaves without a haircut.
}
```

Process scheduling

Scheduling is the method by which task specified by some means is assigned to resources that complete the task. The process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process on the basis of a particular strategy. Process scheduling is an essential part of a Multiprogramming operating systems. Such operating systems allow more than one process to be loaded into the executable memory at a time and the loaded process shares the CPU using time multiplexing.

Objective and criteria of process scheduling

- Max CPU utilization [Keep CPU as busy as possible]
- Fair allocation of CPU.
- Max throughput [Number of processes that complete their execution per time unit]
- Min turnaround time [Time taken by a process to finish execution]
- Min waiting time [Time a process waits in ready queue]
- Min response time [Time when a process produces first response]

4.2 Preemptive vs Non-Preemptive Scheduling

1. Preemptive Scheduling:

Preemptive scheduling is used when a process switches from running state to ready state or from waiting state to ready state. The resources (mainly CPU cycles) are allocated to the process for the limited amount of time and then is taken away, and the process is again placed back in the ready queue if that process still has CPU burst time remaining. That process stays in ready queue till it gets next chance to execute.

Algorithms based on preemptive scheduling are: Round Robin (RR), Shortest Job First (SJF basically non preemptive) and Priority (non-preemptive version), etc.

2. Non-Preemptive Scheduling:

Non-preemptive Scheduling is used when a process terminates, or a process switches from running to waiting state. In this scheduling, once the resources (CPU cycles) is allocated to a process, the process holds the CPU till it gets terminated or it reaches a waiting state. In case of non-preemptive scheduling does not interrupt a process running CPU in middle of the execution. Instead, it waits till the process complete its CPU burst time and then it can allocate the CPU to another process.

Algorithms based on preemptive scheduling are: Shortest Remaining Time First (SRTF), Priority (preemptive version), etc.

Comparison of preemptive and Non preemptive scheduling

BASIS	PREEMPTIVE SCHEDULING	NON PREEMPTIVE SCHEDULING
Basic	The resources are allocated to a process for a limited time.	Once resources are allocated to a process, the process holds it till it completes its burst time or switches to waiting state.
Interrupt	Process can be interrupted in between.	Process cannot be interrupted till it terminates or switches to waiting state.
Starvation	If a high priority process frequently arrives in the ready queue, low priority process may starve.	If a process with long burst time is running CPU, then another process with less CPU burst time may starve.
Overhead	Preemptive scheduling has overheads of scheduling the processes.	Non-preemptive scheduling does not have overheads.
Flexibility	Preemptive scheduling is flexible.	Non-preemptive scheduling is rigid.
Cost	Preemptive scheduling is cost associated.	Non-preemptive scheduling is not cost associative.

4.3 Scheduling Techniques

Different time with respect to a process

- **Arrival Time(AT):** Time at which the process arrives in the ready queue.

- **Burst Time(BT):** Time required by a process for CPU execution.
- **Completion Time(CT):** Time at which process completes its execution.
- **Turn Around Time(TAT):** The time interval from the time of submission of a process to the time of the completion of the process.

OR,

Time Difference between completion time and arrival time. i.e. **TAT= CT - AT**

- **Waiting Time(WT):** The total time waiting in a ready queue.

OR**,**

Time Difference between turnaround time and burst time. i.e.**WT = TAT - BT**

- **Response time(RT):** Amount of time from when a request was submitted until the first response is produced.
- **Quantum size:** It is the specific time interval used to prevent any one process monopolizing the system i.e. can be run exclusively.

A Process Scheduler schedules different processes to be assigned to the CPU based on particular scheduling algorithms**.** The rule or set of rules that is followed to schedule the process is known as process scheduling algorithm**.** The algorithms are either non-preemptive or preemptive**.**

Numerical question are done in class

Lottery Scheduling is a type of process scheduling, somewhat different from other Scheduling. Processes are scheduled in a random manner. Lottery scheduling can be **preemptive or non-preemptive**. It also solves the problem of starvation. Giving each process at least one lottery ticket guarantees that it has a non-zero probability of being selected at each scheduling operation. In this scheduling every process has some tickets and the scheduler picks a random ticket and process having that ticket is the winner and it is executed for a time slice and then another ticket is picked by the scheduler. These tickets represent the share of processes. A process having a higher number of tickets give it more chance to get chosen for execution.

Example – If we have two processes A and B having 60 and 40 tickets respectively out of total of 100 tickets. CPU share of A is 60% and that of B is 40%. These shares are calculated probabilistically and not deterministically.

Explanation:

1. We have two processes A and B. A has 60 tickets (ticket number 1 to 60) and B has 40 tickets (ticket no. 61 to 100).
2. Scheduler picks a random number from 1 to 100. If the picked no. is from 1 to 60 then A is executed otherwise B is executed.

3. An example of 10 tickets picked by the Scheduler may look like this follows:

Ticket number - 73 82 23 45 32 87 49 39 12 09.

Resulting Schedule - B B A A A B A A A A.

1. A is executed 7 times and B is executed 3 times. As you can see that A takes 70% of the CPU and B takes 30% which is not the same as what we need as we need A to have 60% of the CPU and B should have 40% of the CPU. This happens because shares are calculated probabilistically but in a long run(i.e when no. of tickets picked is more than 100 or 1000) we can achieve a share percentage of approx. 60 and 40 for A and B respectively.

Guaranteed Scheduling

- The system tracks how much CPU time each process has consumed.
- It aims to guarantee that each process gets approximately the same share of the total CPU time in a fair manner.
- If a process exceeds its fair share, it will be "inched" or preempted to allow other processes to catch up, ensuring fairness.

Fair Share Calculation

Given nnn processes, each process should get a fraction 1n\frac{1}{n}n1 of the total CPU time. The goal is for each process to have used an equal or nearly equal amount of CPU time by the end.

Example Scenario:

- **Processes**: A,B,CA, B, CA,B,C
- **Current CPU time usage**:
  - AAA has used 2s,
  - BBB has used 3s,
  - CCC has used 1s.
- **Total CPU time so far** = 6s (i.e., 2+3+1=62 + 3 + 1 = 62+3+1=6).
- **Fair share per process** = 13\frac{1}{3}31 of the total time, or **2s** per process.

Scheduling in Real Time System:

Time is crucial and plays an essential role.
eg. the computer in a compact disc player gets the bits as they come off the drive and must convert them into music with a very tight time interval. If the calculation takes too long the music sounds peculiar. Other example includes

- Auto pilot in Aircraft
- Robot control in automated factory.
- Patient monitoringin Factory. (ICU)

Two types:

1. **Hard Real Time system:** There are absolute deadline that must be met.
2. **Soft Real Time system:** Missing an occasional deadline is undesirable but nevertheless tolerable.

In both cases real time behavior is achieved by dividing the program into a no. of processes, each of whose behavior is predictable & known in advance. These processes are short lived and can run to completion. Its the job of schedulers to schedule the process in such a way that all deadlines are met.

If there are m periodic events and event i occurs with period Pi and requires Ci seconds of CPU time to handle each event, then the load can only be handled if

$$\sum_{i=1}^{m} \frac{C_i}{P_i} \leq 1$$

A real-time system that meets this criteria is said to be schedulable.

Thread

A thread is a basic unit of CPU utilization, it comprises a thread ID, a program counter, a register set, and a stack. It shares with other threads belonging to the same process its code section, data section, and other operating system resources, such as open files and signals. A traditional (or heavy weight) process has a single thread of control. If a process has multiple thread of control, it can perform more than one task at a time. Fig below illustrate the difference between single threaded process and a multi- threaded process.

Thread simply enable us to split up a program into logically separate pieces and have the pieces run independently of one another until they need to communicate. In a sense threads are a further level of object orientation for multitasking system.

Multithreading:

Many software package that run on modern desktop pcs are multi-threaded. An application is implemented as a separate process with several threads of control. A web browser might have one thread to display images or text while other thread retrieves data from the network. A word-processor may have a thread for displaying graphics, another thread for reading the character entered by user through the keyboard, and a third thread for performing spelling and grammar checking in the background.

Why Multithreading:

In certain situations, a single application may be required to perform several similar task such as a web server accepts client requests for web pages, images, sound, graphics etc. A busy web server may have several clients concurrently accessing it. So if the web server runs on traditional single threaded process, it would be able to service only one client at a time. The amount of time that the client might have to wait for its request to be serviced is enormous.

One solution of this problem can be thought by creation of new process. When the server receives a new request, it creates a separate process to service that request. But this method is heavy weight. In fact this process creation method was common before threads become popular. Process creation is time consuming and resource intensive. If the new process perform the same task as the existing process, why incur all that overhead? It is generally more efficient for one process that contains multiple threads to serve the same purpose. This approach would multithreaded the web server process. The server would cerate a separate thread that would listen for clients requests. When a request is made, rather than creating another process, it will create a separate thread to service the request.

Benefits of Multi-threading:

Responsiveness: Mutlithreaded interactive application continues to run even if part of it is blocked or performing a lengthy operation, thereby increasing the responsiveness to the user.

Resource Sharing: By default, threads share the memory and the resources of the process to which they belong. It allows an application to have several different threads of activity withing the same address space.

Economy: Allocating memory and resources for process creation is costly. Since thread shares the resources of the process to which they belong, it is more economical to create and context switch threads. It is more time consuming to create and manage process than threads.

Utilization of multiprocessor architecture: The benefits of multi threading can be greatly increased in multiprocessor architecture, where threads may be running in parallel on different processors. Mutlithreading on a multi-CPU increases concurrency.

Multi-Threading Model:

User thread are supported above the kernel and are managed without the kernel support whereas kernel threads are supported and are manged directly by the operating system. Virtually all operating-system includes kernel threads. Ultimately there must exists a relationship between user threads and kernel threads. We have three models for it.

1. Many-to-one model

maps many user level threads to one kernel thread. Thread management is done by the thread library in user space so it is efficient; but the entire process will block if a thread makes a blocking system call. Also only one thread can access the kernel at a time, multiple threads are unable to run in parallel on multiprocessors. Green Threads - a thread library available for Solaris use this model.

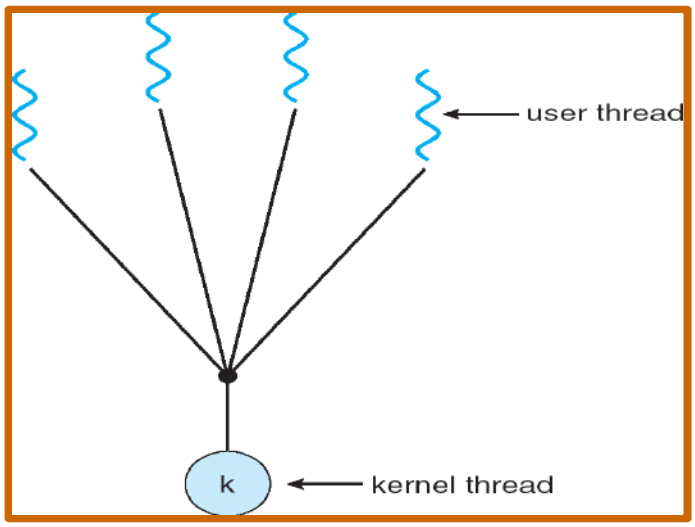


Fig: Many to One threading Model

2. **One-to-one Model:** maps each user thread to a kernel thread. It provides more concurrency than many

to one model by allowing another thread to run when a thread makes a blocking system call. The only drawback to this model is that creating a user thread requires creating the corresponding kernel thread. Linux along with families of Windows operating system use this model.

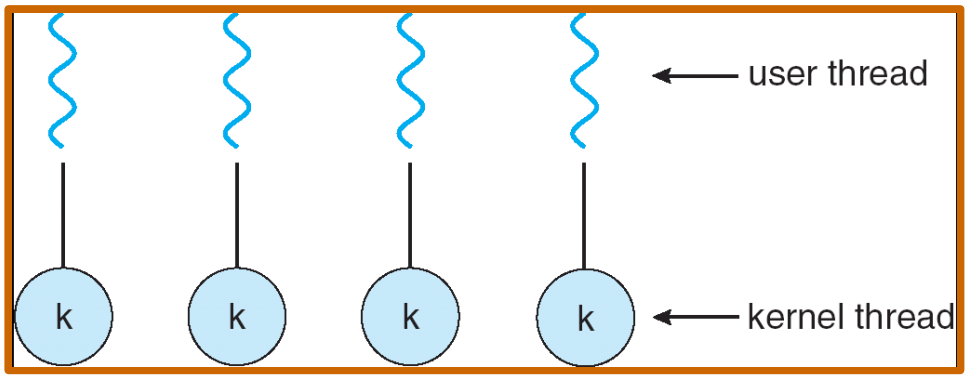
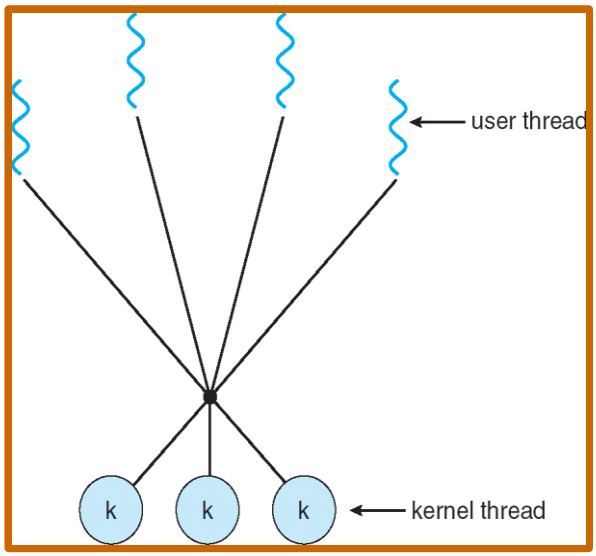


Fig: One to one Threading model

3. Many-to-many Model: multiplexes many user level thread to a smaller or equal number of kernel threads. The number of kernel thread may be specific to either a particular application or a particular machine. Many-to-many model allows the users to create as many threads as he wishes but the true concurrency is not gained because the kernel can schedule only one thread at a time.



Types/ model of Threads

1. **User Level Threads**

- Thread management is done in **user space** of main memory by a **user-level thread library.**
- The user-level or programmer takes the information from thread library which contains the code about creation, destroy, schedule, execution and restoring of threads.
- Kernel don't know about these threads
- User Level Threads are faster to create and manage

Advantages:

1. The most obvious advantage of this technique is that a user-level threads package can be implemented on an Operating System that does not support threads.
2. User-level threads do not require modification to operating systems.
3. **Simple Representation**: Each thread is represented simply by a PC, registers, stack and a small control block, all stored in the user process address space.
4. **Simple Management:** This simply means that creating a thread, switching between threads and synchronization between threads can all be done without intervention of the kernel.

5. Fast and Efficient: Thread switching is not much more expensive than a procedure call.

Disadvantages:

1. User-Level threads are not a perfect solution as with everything else, they are a tradeoff. Since, User-Level threads are invisible to the OS they are not well integrated with the OS. As a result, Os can make poor decisions like scheduling a process with idle threads, blocking a process whose thread initiated an I/O even though the process has other threads that can run and unscheduling a process with a thread holding a lock. Solving this requires communication between between kernel and user-level thread manager.
2. There is a lack of coordination between threads and operating system kernel. Therefore, process as whole gets one time slice irrespect of whether process has one thread or 1000 threads within. It is up to each thread to relinquish control to other threads.
3. User-level threads require non-blocking systems call i.e., a multithreaded kernel. Otherwise, entire process will block in the kernel, even if there are runnable threads left in the processes. For example, if one thread causes a page fault, the process blocks.

2. Kernel Level Threads

- Thread that are defined and managed by directly kernel of Operating system is kernel level threads
- Kernel perform thread creation, scheduling and management
- Kernel threads are used in internal working of operating system
- Kernel threads are slower to create and manage

Advantages:

1. Because kernel has full knowledge of all threads, Scheduler may decide to give more time to a process having large number of threads than process having small number of threads.
2. Kernel-level threads are especially good for applications that frequently block.

Disadvantages:

1. The kernel-level threads are slow and inefficient. For instance, threads operations are hundreds of times slower than that of user-level threads.
2. Since kernel must manage and schedule threads as well as processes. It requires a full thread control block (TCB) for each thread to maintain information about threads. As a result, there is significant overhead and increased in kernel complexity.

P4 - Difference between thread and process?

	Process	Threads
1	Process means any program is in execution	Thread means a segment of a process
2	Process takes more time for creation • Process creation is "expensive", because OS has to set up a complete new virtual memory space for the process. "expensive" means takes a lot of CPU time.	Thread takes lass time for creation • Thread run in the address space of the parent process. OS does not need to create separate Virtual address space for thread
3	Process takes more time to terminate **How and why ?**	Thread takes less time to terminate **How and Why ?**
4	A context switch involve switching the stack pointer and program counter, restoring other registers and **switching the MMU context** as well.	a context switch is simply a case of switching the stack pointer and program counter and restoring other registers only
6	IPC mechanism is required for communication between two Process	Threads can interact without the need of inter-process communication.
7	Each process is independent	Thread cannot run independent
8	Only code segment can be shared between two processes if they are spawned from same binary.	Threads has its own stack , program counter and set of register **but it shares code, data and File descriptor**